# Most Asked JavaScript Interview Questions

# By Skill Foundry

Copyright © 2025 Skill Foundry

**1. What is JavaScript and how is it different from Java?**

**Answer:** JavaScript is a high-level, interpreted programming language primarily used for web development. It is lightweight, dynamic, and follows a prototype-based object-oriented model. Unlike Java, which is a statically-typed, compiled language used for general-purpose programming, JavaScript is dynamically typed and interpreted, making it more flexible for front-end and back-end development.

---

**2. Explain the difference between let, const, and var.**

**Answer:**

- var: Function-scoped, can be redeclared and updated, hoisted with undefined.
- let: Block-scoped, cannot be redeclared within the same scope, hoisted without initialization.
- const: Block-scoped, cannot be reassigned, must be initialized during declaration.

---

**3. What is the difference between == and === in JavaScript?**

**Answer:**

- == (loose equality): Compares values after type conversion. Example: 5 == "5" returns true.
- === (strict equality): Compares both value and type without conversion. Example: 5 === "5" returns false.

---

**4. What is hoisting in JavaScript?**

**Answer:**
Hoisting is JavaScript's default behavior of moving function and variable declarations to the top of their scope before execution. However, variables

Copyright © 2025 Skill Foundry

declared with let and const are hoisted but not initialized, causing a ReferenceError if accessed before declaration.

---

**5. What is a closure in JavaScript?**

**Answer:**
A closure is a function that retains access to its parent's scope even after the parent function has executed.
Example:

```
function outer() {
    let count = 0;
    return function inner() {
        count++;
        console.log(count);
    };
}
const counter = outer();
counter(); // 1
counter(); // 2
```

Here, inner retains access to count even after outer has finished execution.

---

**6. What is the difference between null and undefined?**

**Answer:**

- null: Explicitly assigned to a variable to indicate no value.
- undefined: A variable that has been declared but not assigned any value.

Example:

```
let a; // undefined
let b = null; // null
```

Copyright © 2025 Skill Foundry

---

**7. What are arrow functions and how are they different from regular functions?**

**Answer:**
Arrow functions (=>) provide a concise syntax for function expressions. Differences from regular functions:

- **No this binding:** Arrow functions inherit this from their surrounding scope.
- **Cannot be used as constructors:** Calling new on an arrow function results in an error.
- **No arguments object:** Cannot access arguments directly.

Example:

```
const add = (a, b) => a + b;

console.log(add(2, 3)); // 5
```

---

**8. What is the difference between synchronous and asynchronous JavaScript?**

**Answer:**

- **Synchronous:** Executes code sequentially, blocking further execution until the current task completes.
- **Asynchronous:** Uses callbacks, promises, or async/await to execute tasks without blocking the main thread.

Example:

```
console.log("Start");

setTimeout(() => console.log("Async Task"),
1000);

console.log("End");


// Output:
```

Copyright © 2025 Skill Foundry

```
// Start
// End
// Async Task (after 1 sec)
```

---

**9. What is the event loop in JavaScript?**

**Answer:**
The event loop is a mechanism that handles asynchronous operations. It continuously checks the call stack and the message queue, executing tasks in the call stack before processing queued asynchronous callbacks.

---

**10. What are promises in JavaScript?**

**Answer:**
A promise represents a value that may be available now, later, or never. It has three states:

- **Pending**: Initial state, result is unknown.
- **Fulfilled**: Operation completed successfully.
- **Rejected**: Operation failed.

Example:

```
const myPromise = new Promise((resolve, reject)
=> {
    setTimeout(() => resolve("Success!"), 2000);
});
myPromise.then(result => console.log(result)); //
Logs "Success!" after 2 seconds
```

Copyright © 2025 Skill Foundry

**11. What is the difference between function declaration and function expression?**

**Answer:**

- **Function Declaration:** Named function that can be hoisted.

```
function greet() {
    console.log("Hello");
}
```

- **Function Expression:** Assigned to a variable, not hoisted.

```
const greet = function() {
    console.log("Hello");
};
```

---

**12. What is the difference between call(), apply(), and bind()?**

**Answer:**

- **call()**: Calls a function with a given this value and arguments passed individually.

```
function greet(name) {
    console.log(`Hello, ${name}`);
}
greet.call(null, "John"); // Hello, John
```

- **apply()**: Similar to call(), but arguments are passed as an array.

```
greet.apply(null, ["John"]); // Hello, John
```

- **bind()**: Returns a new function with this permanently bound.

```
const greetJohn = greet.bind(null, "John");
greetJohn(); // Hello, John
```

---

Copyright © 2025 Skill Foundry

**13. What is the difference between shallow copy and deep copy in JavaScript?**

**Answer:**

- **Shallow Copy**: Copies only references, not nested objects.

```
let obj1 = { a: 1, b: { c: 2 } };
let obj2 = { ...obj1 };
obj2.b.c = 42; // Affects obj1 as well
```

- **Deep Copy**: Creates a completely new object.

```
let obj3 = JSON.parse(JSON.stringify(obj1)); // Deep copy
obj3.b.c = 99; // Does not affect obj1
```

---

**14. What are higher-order functions in JavaScript?**

**Answer:**
A higher-order function takes another function as an argument or returns a function.
Example:

```
function multiplyBy(num) {
    return function(x) {
        return x * num;
    };
}
const double = multiplyBy(2);
console.log(double(5)); // 10
```

---

Copyright © 2025 Skill Foundry

**15. What is the difference between map(), filter(), and reduce()?**

**Answer:**

- **map()**: Transforms each array element.

```
let arr = [1, 2, 3].map(x => x * 2); // [2, 4, 6]
```

- **filter()**: Returns a subset based on condition.

```
let arr = [1, 2, 3, 4].filter(x => x % 2 === 0);
// [2, 4]
```

- **reduce()**: Reduces array to a single value.

```
let sum = [1, 2, 3].reduce((acc, x) => acc + x,
0); // 6
```

---

**16. What is memoization in JavaScript?**

**Answer:**
Memoization is an optimization technique that caches function results.
Example:

```
function memoizedFactorial() {
    let cache = {};
    return function factorial(n) {
        if (n in cache) return cache[n];
        if (n === 0) return 1;
        return cache[n] = n * factorial(n - 1);
    };
}
const fact = memoizedFactorial();
console.log(fact(5)); // 120 (cached result)
```

---

Copyright © 2025 Skill Foundry

### 17. What is event delegation in JavaScript?

**Answer:**
Event delegation is a technique where a parent handles events for dynamically added child elements using event bubbling.
Example:

```
document.getElementById("parent").addEventListene
r("click", function(event) {
    if (event.target.matches(".child")) {
        console.log("Child clicked!");
    }
});
```

---

### 18. What are template literals in JavaScript?

**Answer:**
Template literals allow embedded expressions using backticks (`).
Example:

```
let name = "John";
console.log(`Hello, ${name}!`); // Hello, John!
```

---

### 19. What is destructuring in JavaScript?

**Answer:**
Destructuring allows extracting values from objects or arrays easily.
Example:

```
const obj = { name: "Alice", age: 25 };
const { name, age } = obj;
console.log(name, age); // Alice 25
```

---

Copyright © 2025 Skill Foundry

### 20. What are default parameters in JavaScript?

**Answer:**
Default parameters allow function parameters to have default values.
Example:

```
function greet(name = "Guest") {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Guest!
```

### 21. What is the difference between setTimeout and setInterval?

**Answer:**

- **setTimeout(callback, delay)**: Executes the callback once after a specified delay.
- **setInterval(callback, interval)**: Executes the callback repeatedly at the specified interval.

Example:

```
setTimeout(() => console.log("Executed after 2 seconds"), 2000);


setInterval(() => console.log("Executed every 2 seconds"), 2000);
```

---

### 22. What is the difference between deep copy and shallow copy in JavaScript?

**Answer:**

- **Shallow Copy**: Copies object references, so changes affect both objects.

```
let obj1 = { a: 1, b: { c: 2 } };
let obj2 = { ...obj1 }; // Shallow copy
```

Copyright © 2025 Skill Foundry

```
obj2.b.c = 42; // Changes obj1.b.c as well
```

- **Deep Copy**: Creates a completely independent copy.

```
let obj3 = JSON.parse(JSON.stringify(obj1)); // Deep copy
obj3.b.c = 99; // Does not affect obj1
```

---

**23. How does JavaScript handle asynchronous operations?**

**Answer:**
JavaScript uses the **event loop**, **callbacks**, **promises**, and **async/await** for asynchronous operations.
Example using promises:

```
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

---

**24. What is a generator function in JavaScript?**

**Answer:**
A generator function (function*) returns an iterator and can pause execution using yield.
Example:

```
function* numberGenerator() {
    yield 1;
    yield 2;
    yield 3;
}
const gen = numberGenerator();
console.log(gen.next().value); // 1
```

Copyright © 2025 Skill Foundry

```
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

---

**25. What is the purpose of Object.freeze() and Object.seal()?**

**Answer:**

- **Object.freeze()**: Prevents adding, deleting, or modifying properties.
- **Object.seal()**: Prevents adding or deleting properties but allows modification of existing ones.

Example:

```
const obj = { a: 1 };
Object.freeze(obj);
obj.a = 2; // No effect

const obj2 = { b: 1 };
Object.seal(obj2);
obj2.b = 2; // Works
delete obj2.b; // Fails
```

---

**26. What is the purpose of this keyword in JavaScript?**

**Answer:**
this refers to the object that is executing the function. Its value depends on how a function is called:

- In **global scope**, this refers to window (or global in Node.js).
- In **object methods**, this refers to the object itself.
- In **event handlers**, this refers to the element receiving the event.

Copyright © 2025 Skill Foundry

Example:

```
const obj = {
    name: "Alice",
    greet: function() {
        console.log(this.name);
    }
};
obj.greet(); // Alice
```

---

**27. What is the purpose of new keyword in JavaScript?**

**Answer:**
The new keyword is used to create instances of objects from a constructor function.
Example:

```
function Person(name) {
    this.name = name;
}
const p1 = new Person("John");
console.log(p1.name); // John
```

---

**28. How do you check if a variable is an array in JavaScript?**

**Answer:**
Use Array.isArray() to check if a variable is an array.
Example:

```
console.log(Array.isArray([1, 2, 3])); // true
console.log(Array.isArray({ a: 1 })); // false
```

---

Copyright © 2025 Skill Foundry

**29. What are ES6 modules and how do you use them?**

**Answer:**
ES6 modules allow code to be split into reusable files using export and import.
Example:

**File: math.js**

```
export function add(a, b) {
    return a + b;
}
```

**File: app.js**

```
import { add } from './math.js';
console.log(add(2, 3)); // 5
```

---

**30. What is the purpose of try...catch in JavaScript?**

**Answer:**
try...catch is used for error handling in JavaScript.
Example:

```
try {
    let result = x / 0;
} catch (error) {
    console.log("Error occurred:",
error.message);
}
```

**31. What is the difference between synchronous and asynchronous code in JavaScript?**

**Answer:**

- **Synchronous code**: Executes line by line and blocks further execution until the current task is completed.

Copyright © 2025 Skill Foundry

```
console.log("Start");

console.log("End");
```

- **Asynchronous code**: Executes tasks in the background, allowing other code to run without waiting.

```
console.log("Start");

setTimeout(() => console.log("Async task
completed"), 1000);

console.log("End");

Output:

sql


Start

End

Async task completed
```

---

**32. What is an Immediately Invoked Function Expression (IIFE)?**

**Answer:**
An IIFE is a function that runs immediately after being defined. It helps avoid polluting the global scope.
Example:

```
(function() {

    console.log("This runs immediately!");

})();
```

---

**33. What is the difference between == and === in JavaScript?**

**Answer:**

- **== (loose equality)**: Converts types before comparing.

Copyright © 2025 Skill Foundry

```
console.log(5 == "5"); // true
```

- **=== (strict equality)**: Checks both value and type.

```
console.log(5 === "5"); // false
```

---

**34. What is the typeof operator in JavaScript?**

**Answer:**
The typeof operator returns the type of a variable.
Example:

```
console.log(typeof 42); // "number"
console.log(typeof "hello"); // "string"
console.log(typeof {}); // "object"
console.log(typeof undefined); // "undefined"
```

---

**35. How do you create a class in JavaScript?**

**Answer:**
ES6 introduced the class keyword for object-oriented programming.
Example:

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    greet() {
        console.log(`Hello, my name is
${this.name}`);
    }
}
```

Copyright © 2025 Skill Foundry

```
const p = new Person("Alice", 30);
p.greet(); // Hello, my name is Alice
```

---

**36. What is the difference between null and undefined?**

**Answer:**

- **null**: A value assigned to indicate "no value".
- **undefined**: A variable that has been declared but not assigned a value. Example:

```
let x; // undefined
let y = null; // null
```

---

**37. What is closure in JavaScript?**

**Answer:**
A closure is a function that retains access to its parent scope, even after the parent function has executed.
Example:

```
function outer() {
    let count = 0;
    return function inner() {
        count++;
        console.log(count);
    };
}
const counter = outer();
counter(); // 1
counter(); // 2
```

---

Copyright © 2025 Skill Foundry

**38. What is the difference between let, const, and var?**

**Answer:**

- **var**: Function-scoped, can be redeclared, and hoisted.
- **let**: Block-scoped, cannot be redeclared.
- **const**: Block-scoped, cannot be redeclared or reassigned.
  Example:

```
var a = 10; // Function-scoped
let b = 20; // Block-scoped
const c = 30; // Cannot be reassigned
```

---

**39. What is the difference between arrow functions and regular functions?**

**Answer:**

- **Arrow functions**: Do not have their own this, making them useful in callbacks.
- **Regular functions**: Have their own this.
  Example:

```
const obj = {
    value: 10,
    regularFunc: function() {
        console.log(this.value);
    },
    arrowFunc: () => {
        console.log(this.value);
    }
};
obj.regularFunc(); // 10
```

Copyright © 2025 Skill Foundry

```
obj.arrowFunc(); // undefined (because arrow
functions do not have their own `this`)
```

---

**40. What is the difference between a promise and async/await?**

**Answer:**

- **Promises**: Handle async operations with .then() and .catch().

```
fetch("https://api.example.com")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

- **async/await**: Makes async code look synchronous and is easier to read.

```
async function fetchData() {
    try {
        let response = await
fetch("https://api.example.com");
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.error(error);
    }
}
fetchData();
```

**41. What is the event loop in JavaScript?**

Copyright © 2025 Skill Foundry

**Answer:**
The event loop is a mechanism that allows JavaScript to handle asynchronous operations by continuously checking the call stack and the task queue.
Example:

```
console.log("Start");

setTimeout(() => console.log("Timeout"), 0);

console.log("End");
```

**Output:**

```sql

Start
End
Timeout
```

Even though setTimeout has a 0ms delay, it runs after synchronous code due to the event loop.

---

**42. What is debouncing in JavaScript?**

**Answer:**
Debouncing ensures that a function is executed only after a specified delay, preventing excessive function calls.
Example:

```
function debounce(func, delay) {
    let timer;
    return function(...args) {
        clearTimeout(timer);
```

Copyright © 2025 Skill Foundry

```
        timer = setTimeout(() => func(...args), delay);
    };
}
window.addEventListener("resize", debounce(() => console.log("Resized!"), 300));
```

The function will execute only after 300ms of inactivity.

---

**43. What is throttling in JavaScript?**

**Answer:**
Throttling ensures a function is executed at most once in a given time interval.
Example:

```
function throttle(func, limit) {
    let lastCall = 0;
    return function(...args) {
        let now = Date.now();
        if (now - lastCall >= limit) {
            lastCall = now;
            func(...args);
        }
    };
}
window.addEventListener("scroll", throttle(() => console.log("Scrolling!"), 1000));
```

This ensures the function runs at most once per second.

---

**44. What is the difference between map(), forEach(), and filter()?**

Copyright © 2025 Skill Foundry

**Answer:**

- **map()**: Returns a new array with transformed values.
- **forEach()**: Iterates over an array but does not return a new array.
- **filter()**: Returns a new array with only elements that pass the condition.

Example:

```
let nums = [1, 2, 3, 4];


console.log(nums.map(n => n * 2)); // [2, 4, 6, 8]
nums.forEach(n => console.log(n * 2)); // Prints 2, 4, 6, 8
console.log(nums.filter(n => n % 2 === 0)); // [2, 4]
```

---

**45. What is the difference between call(), apply(), and bind()?**

**Answer:**

- **call()**: Invokes a function with a specified this and arguments.
- **apply()**: Same as call(), but arguments are passed as an array.
- **bind()**: Returns a new function with this permanently bound.

Example:

```
function greet(greeting) {
    console.log(`${greeting}, ${this.name}`);
}


const user = { name: "Alice" };


greet.call(user, "Hello");  // Hello, Alice
```

Copyright © 2025 Skill Foundry

```
greet.apply(user, ["Hi"]);  // Hi, Alice
const boundGreet = greet.bind(user, "Hey");
boundGreet();  // Hey, Alice
```

---

**46. What is memoization in JavaScript?**

**Answer:**
Memoization is an optimization technique that caches function results to avoid redundant computations.
Example:

```
function memoize(fn) {
    let cache = {};
    return function(n) {
        if (n in cache) {
            return cache[n];
        }
        cache[n] = fn(n);
        return cache[n];
    };
}

const factorial = memoize(n => (n <= 1 ? 1 : n *
factorial(n - 1)));
```

```
console.log(factorial(5)); // Computed and stored
console.log(factorial(5)); // Retrieved from
cache
```

---

Copyright © 2025 Skill Foundry

**47. What is the difference between localStorage, sessionStorage, and cookies?**

**Answer:**

| Feature | localStorage | sessionStorage | cookies |
|---|---|---|---|
| Data lifetime | Forever (unless manually cleared) | Until the session ends (browser closed) | Can have expiration dates |
| Storage limit | ~5MB | ~5MB | ~4KB |
| Accessible by server? | No | No | Yes |

Example:

```
localStorage.setItem("key", "value");
console.log(localStorage.getItem("key")); // "value"
```

---

**48. How does JavaScript handle errors using try...catch...finally?**

**Answer:**
JavaScript provides error handling with try...catch and an optional finally block.
**Example**:

```
try {
    let result = 10 / 0;
    console.log(result);
} catch (error) {
    console.log("An error occurred:", error.message);
} finally {
    console.log("Execution completed.");
}
```

Copyright © 2025 Skill Foundry

---

**49. What are template literals in JavaScript?**

**Answer:**
Template literals use backticks (`) to allow multi-line strings and string interpolation.
Example:

```
let name = "Alice";
console.log(`Hello, ${name}!`);
```

---

**50. What is destructuring in JavaScript?**

**Answer:**
Destructuring allows extracting values from arrays or objects into separate variables.
Example:

```
// Array destructuring
let [a, b] = [1, 2];
console.log(a, b); // 1 2

// Object destructuring
let person = { name: "Alice", age: 25 };
let { name, age } = person;
console.log(name, age); // Alice 25
```

**51. What is the difference between shallow copy and deep copy in JavaScript?**

**Answer:**

- **Shallow Copy**: Copies only references of nested objects, not the actual values.

Copyright © 2025 Skill Foundry

- **Deep Copy**: Copies all values, creating a completely independent copy.

Example:

```
// Shallow copy using spread operator
let obj1 = { a: 1, b: { c: 2 } };
let obj2 = { ...obj1 };
obj2.b.c = 42;
console.log(obj1.b.c); // 42 (affected)

// Deep copy using JSON
let obj3 = JSON.parse(JSON.stringify(obj1));
obj3.b.c = 99;
console.log(obj1.b.c); // 42 (not affected)
```

---

**52. What is the difference between function declaration and function expression?**

**Answer:**

- **Function Declaration**: Defined with the function keyword and hoisted.
- **Function Expression**: Assigned to a variable and not hoisted.

Example:

```
// Function Declaration
function greet() {
    console.log("Hello!");
}
greet(); // Works
```

Copyright © 2025 Skill Foundry

```
// Function Expression
const greetFunc = function() {
    console.log("Hello!");
};
greetFunc(); // Works
```

---

**53. What is the purpose of the new keyword in JavaScript?**

**Answer:**
The new keyword is used to create instances of a constructor function.
Example:

```
function Person(name) {
    this.name = name;
}
const p = new Person("Alice");
console.log(p.name); // Alice
```

---

**54. What is the this keyword in JavaScript?**

**Answer:**
this refers to the object that calls the function. Its value depends on how the function is called.
Example:

```
const obj = {
    name: "Alice",
    greet: function() {
        console.log(this.name);
    }
};
```

Copyright © 2025 Skill Foundry

```
obj.greet(); // Alice
```

For arrow functions, this is inherited from the surrounding scope.

---

**55. How does setTimeout work with a loop?**

**Answer:**
When using setTimeout inside a loop, closures can cause unexpected behavior.
Example:

```
for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 1000);
}
// Output: 3 3 3 (because `var` is function-scoped)
To fix this, use let:
javascript

for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 1000);
}
// Output: 0 1 2
```

---

**56. What is Object.freeze() and Object.seal()?**

**Answer:**

- **Object.freeze()**: Prevents adding, deleting, or modifying properties.
- **Object.seal()**: Prevents adding or deleting properties but allows modification of existing ones.

Copyright © 2025 Skill Foundry

Example:

```
let obj = { name: "Alice" };
Object.freeze(obj);
obj.name = "Bob"; // No effect
console.log(obj.name); // Alice

let obj2 = { age: 25 };
Object.seal(obj2);
obj2.age = 30; // Allowed
delete obj2.age; // Not allowed
```

---

**57. What is the in operator in JavaScript?**

**Answer:**
The in operator checks if a property exists in an object.
Example:

```
let obj = { name: "Alice" };
console.log("name" in obj); // true
console.log("age" in obj); // false
```

---

**58. How do you check if an object is empty in JavaScript?**

**Answer:**
Using Object.keys():

```
let obj = {};
console.log(Object.keys(obj).length === 0); //
true
```

---

Copyright © 2025 Skill Foundry

**59. What is hasOwnProperty() in JavaScript?**

**Answer:**
It checks if an object has a specific property (excluding prototype properties).
Example:

```
let obj = { name: "Alice" };
console.log(obj.hasOwnProperty("name")); // true
console.log(obj.hasOwnProperty("age")); // false
```

---

**60. What is the difference between push() and unshift() in arrays?**

**Answer:**

- **push()**: Adds elements to the **end** of an array.
- **unshift()**: Adds elements to the **beginning** of an array.

Example:

```
let arr = [1, 2, 3];
arr.push(4);
console.log(arr); // [1, 2, 3, 4]

arr.unshift(0);
console.log(arr); // [0, 1, 2, 3, 4]
```

**61. What is the difference between pop() and shift() in arrays?**

**Answer:**

- **pop()**: Removes the last element from an array and returns it.
- **shift()**: Removes the first element from an array and returns it.

Example:

Copyright © 2025 Skill Foundry

```
let arr = [1, 2, 3, 4];
console.log(arr.pop()); // 4
console.log(arr); // [1, 2, 3]

console.log(arr.shift()); // 1
console.log(arr); // [2, 3]
```

---

**62. What is the difference between slice() and splice()?**

**Answer:**

- **slice(start, end)**: Returns a new array without modifying the original.
- **splice(start, deleteCount, items...)**: Modifies the original array by adding/removing elements.

Example:

```
let arr = [1, 2, 3, 4, 5];

console.log(arr.slice(1, 3)); // [2, 3] (new array)
console.log(arr); // [1, 2, 3, 4, 5] (unchanged)

arr.splice(1, 2, 9, 10);
console.log(arr); // [1, 9, 10, 4, 5]
```

---

**63. What is the difference between == and === in JavaScript?**

**Answer:**

- **== (Loose equality)**: Converts values to a common type before comparing.
- **=== (Strict equality)**: Compares both value and type.

Copyright © 2025 Skill Foundry

Example:

```
console.log(5 == "5");  // true (type conversion occurs)
console.log(5 === "5"); // false (different types)
```

---

**64. What is the difference between null and undefined?**

**Answer:**

- **null**: A value that represents "no value" and is assigned explicitly.
- **undefined**: A variable that has been declared but not assigned a value.

Example:

```
let a;
console.log(a); // undefined

let b = null;
console.log(b); // null
```

---

**65. What is hoisting in JavaScript?**

**Answer:**
Hoisting moves function and variable declarations to the top of their scope before execution.

Example:

```
console.log(x); // undefined
var x = 5;

sayHello(); // Works
function sayHello() {
```

Copyright © 2025 Skill Foundry

```
    console.log("Hello!");
}
```

Only declarations are hoisted, not initializations.

---

**66. What are higher-order functions in JavaScript?**

**Answer:**
A higher-order function takes another function as an argument or returns a function.

Example:

```
function operate(a, b, func) {
    return func(a, b);
}

console.log(operate(5, 3, (x, y) => x + y)); // 8
```

---

**67. What is function currying in JavaScript?**

**Answer:**
Currying transforms a function that takes multiple arguments into a series of functions, each taking one argument.

Example:

```
function curry(a) {
    return function(b) {
        return function(c) {
            return a + b + c;
        };
    };
}
```

Copyright © 2025 Skill Foundry

```
console.log(curry(2)(3)(4)); // 9
```

---

**68. What is the difference between synchronous and asynchronous code?**

**Answer:**

- **Synchronous**: Executes line by line, blocking further execution until a task completes.
- **Asynchronous**: Doesn't block execution; tasks run in the background.

Example:

```
console.log("Start");


setTimeout(() => console.log("Async Task"),
1000);


console.log("End");
```

Output:

```
sql


Start

End

Async Task
```

---

**69. What is a closure in JavaScript?**

**Answer:**
A closure is a function that retains access to variables from its outer scope even after the outer function has executed.

Example:

Copyright © 2025 Skill Foundry

```
function outer() {
    let count = 0;
    return function inner() {
        count++;
        console.log(count);
    };
}

const counter = outer();
counter(); // 1
counter(); // 2
```

---

**70. What is the Promise object in JavaScript?**

**Answer:**
A Promise represents a value that may be available now, later, or never.

Example:

```
let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("Success!"), 1000);
});

promise.then(res => console.log(res)).catch(err
=> console.log(err));
```

**Output (after 1 second):**

```
Success!
```

Copyright © 2025 Skill Foundry

**71. What are the different states of a JavaScript Promise?**

**Answer:**
A Promise has three states:

- **Pending**: Initial state, neither resolved nor rejected.
- **Fulfilled**: The operation was successful.
- **Rejected**: The operation failed.

Example:

```
let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("Success!"), 1000);
});

promise.then(res => console.log(res)).catch(err
=> console.log(err));
```

---

**72. What is the difference between .then() and .catch() in Promises?**

**Answer:**

- **.then()**: Handles the success case.
- **.catch()**: Handles errors or rejected cases.

Example:

```
let promise = new Promise((resolve, reject) => {
    let success = false;
    success ? resolve("Success!") :
reject("Error!");
});

promise
```

Copyright © 2025 Skill Foundry

```
    .then(res => console.log(res)) // Runs if
resolved
    .catch(err => console.log(err)); // Runs if
rejected
```

---

**73. What is async and await in JavaScript?**

**Answer:**
async makes a function return a Promise, and await pauses execution until the Promise resolves.

Example:

```
async function fetchData() {
    let response = await
fetch("https://jsonplaceholder.typicode.com/todos
/1");
    let data = await response.json();
    console.log(data);
}

fetchData();
```

---

**74. How do you handle errors in async/await?**

**Answer:**
Use try...catch inside an async function.

Example:

```
async function fetchData() {
    try {
        let response = await
fetch("https://invalid-url.com");
```

Copyright © 2025 Skill Foundry

```
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.log("Error:", error.message);
    }
}

fetchData();
```

---

**75. What is event delegation in JavaScript?**

**Answer:**
Event delegation allows a parent element to handle events for dynamically added child elements.

Example:

```
document.getElementById("parent").addEventListene
r("click", function(event) {
    if (event.target.tagName === "BUTTON") {
        console.log("Button clicked:",
event.target.innerText);
    }
});
```

---

**76. What is debouncing in JavaScript?**

**Answer:**
Debouncing limits the rate at which a function executes by delaying it until after a pause in execution.

Example:

```
function debounce(func, delay) {
```

Copyright © 2025 Skill Foundry

```
    let timer;
    return function(...args) {
        clearTimeout(timer);
        timer = setTimeout(() => func(...args),
delay);
    };
}

const handleResize = debounce(() =>
console.log("Resized!"), 500);
window.addEventListener("resize", handleResize);
```

---

**77. What is throttling in JavaScript?**

**Answer:**
Throttling ensures a function executes at most once in a specified time period.

Example:

```
function throttle(func, limit) {
    let lastCall = 0;
    return function(...args) {
        let now = Date.now();
        if (now - lastCall >= limit) {
            lastCall = now;
            func(...args);
        }
    };
}
```

Copyright © 2025 Skill Foundry

```
const handleScroll = throttle(() =>
console.log("Scrolled!"), 1000);

window.addEventListener("scroll", handleScroll);
```

---

**78. What is the difference between debouncing and throttling?**

**Answer:**

- **Debouncing**: Delays execution until after a pause.
- **Throttling**: Executes at fixed intervals, ignoring extra calls.

Example:

- **Debounce**: Useful for input fields.
- **Throttle**: Useful for scroll and resize events.

---

**79. What is the localStorage API?**

**Answer:**
localStorage allows storing key-value pairs persistently in the browser.

Example:

```
localStorage.setItem("username", "JohnDoe");

console.log(localStorage.getItem("username")); //
JohnDoe

localStorage.removeItem("username");
```

---

**80. What is the difference between localStorage, sessionStorage, and cookies?**

**Answer:**

| Storage Type | Lifetime | Size Limit | Accessible from Server? |
|---|---|---|---|
| localStorage | Permanent | ~5MB | No |

Copyright © 2025 Skill Foundry

| Storage Type | Lifetime | Size Limit | Accessible from Server? |
|---|---|---|---|
| sessionStorage | Until tab closes | ~5MB | No |
| cookies | Custom expiry | ~4KB | Yes |

Example:

```
sessionStorage.setItem("key", "value"); //
Temporary storage
```

**81. What is the difference between map(), filter(), and reduce() in JavaScript?**

**Answer:**

- **map()**: Transforms each element in an array and returns a new array.
- **filter()**: Returns a new array with elements that meet a condition.
- **reduce()**: Accumulates array values into a single value.

Example:

```
let numbers = [1, 2, 3, 4, 5];


let squared = numbers.map(n => n * n); // [1, 4,
9, 16, 25]

let evens = numbers.filter(n => n % 2 === 0); //
[2, 4]

let sum = numbers.reduce((acc, curr) => acc +
curr, 0); // 15
```

---

**82. What is the difference between forEach() and map()?**

**Answer:**

- **forEach()**: Iterates over an array but doesn’t return a new array.
- **map()**: Iterates and returns a new array with transformed elements.

Copyright © 2025 Skill Foundry

Example:

```
let arr = [1, 2, 3];


arr.forEach((num, index) => arr[index] = num *
2); // Modifies original array
```

```
let doubled = arr.map(num => num * 2); // Returns
a new array
```

---

**83. What is object destructuring in JavaScript?**

**Answer:**
Object destructuring allows extracting properties from an object into variables.

Example:

```
const user = { name: "Alice", age: 25 };


const { name, age } = user;
console.log(name, age); // Alice 25
```

---

**84. What is array destructuring in JavaScript?**

**Answer:**
Array destructuring allows extracting elements from an array into variables.

Example:

```
const arr = [10, 20, 30];


const [first, second] = arr;
console.log(first, second); // 10 20
```

---

Copyright © 2025 Skill Foundry

**85. What is the spread operator (...) in JavaScript?**

**Answer:**
The spread operator expands an array/object into individual elements.

Example:

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2,
c: 3 }
```

---

**86. What is the rest parameter (...) in JavaScript?**

**Answer:**
The rest parameter collects multiple arguments into an array.

Example:

```
function sum(...numbers) {
    return numbers.reduce((acc, num) => acc +
num, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

---

**87. What is optional chaining (?.) in JavaScript?**

**Answer:**
Optional chaining prevents errors when accessing deeply nested properties.

Example:

Copyright © 2025 Skill Foundry

```
let user = { profile: { name: "John" } };


console.log(user.profile?.name); // John
console.log(user.address?.city); // undefined (no
error)
```

---

**88. What is nullish coalescing (??) in JavaScript?**

**Answer:**
?? returns the right-hand value only if the left-hand value is null or undefined.

Example:

```
let user = null;
console.log(user ?? "Guest"); // "Guest"


let name = "";
console.log(name ?? "Anonymous"); // "" (because
it's not null/undefined)
```

---

**89. What is setTimeout and setInterval in JavaScript?**

**Answer:**

- **setTimeout(func, delay)**: Executes a function after a specified delay.
- **setInterval(func, interval)**: Repeats execution at a specified interval.

Example:

```
setTimeout(() => console.log("Delayed
execution"), 2000); // Executes once after 2s


let interval = setInterval(() =>
console.log("Repeating task"), 1000);
```

Copyright © 2025 Skill Foundry

```
setTimeout(() => clearInterval(interval), 5000);
// Stops after 5s
```

---

**90. How do you create a deep copy of an object in JavaScript?**

**Answer:**
Use JSON.parse(JSON.stringify(obj)) or the structured clone API.

Example:

```
let obj = { a: 1, b: { c: 2 } };
let deepCopy = JSON.parse(JSON.stringify(obj));


deepCopy.b.c = 99;
console.log(obj.b.c); // 2 (original object
remains unchanged)
```

**91. What is the difference between == and === in JavaScript?**

**Answer:**

- == (loose equality) compares values but performs type coercion.
- === (strict equality) compares both value and type.

Example:

```
console.log(5 == "5");  // true (because of type
conversion)
console.log(5 === "5"); // false (different
types)
```

---

**92. What are truthy and falsy values in JavaScript?**

**Answer:**

- **Truthy values**: Any value that evaluates to true in a boolean context.

Copyright © 2025 Skill Foundry

- **Falsy values**: Values that evaluate to false.

Falsy values in JavaScript:

false, 0, -0, "", null, undefined, NaN

Example:

```
if ("hello") console.log("Truthy"); // This runs
if (0) console.log("Falsy"); // This doesn't run
```

---

**93. What is the purpose of the typeof operator?**

**Answer:**
The typeof operator returns the type of a variable as a string.

Example:

```
console.log(typeof 42);       // "number"
console.log(typeof "hello");  // "string"
console.log(typeof true);     // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);     // "object" (a
known JavaScript quirk)
```

---

**94. What is the difference between null and undefined?**

**Answer:**

- null: Represents an intentional absence of value.
- undefined: Represents an uninitialized variable or missing property.

Example:

```
let a = null;
```

Copyright © 2025 Skill Foundry

```
let b;
console.log(a); // null
console.log(b); // undefined
```

---

**95. What is the difference between function expressions and function declarations?**

**Answer:**

- **Function Declaration**: Can be hoisted.
- **Function Expression**: Assigned to a variable and not hoisted.

Example:

javascript

```
// Function Declaration
function greet() {
    console.log("Hello");
}

// Function Expression
const greetExp = function() {
    console.log("Hello");
};
```

---

**96. What are JavaScript generators?**

**Answer:**
Generators allow pausing and resuming execution using yield.

Example:

Copyright © 2025 Skill Foundry

```
function* generator() {
    yield 1;
    yield 2;
    yield 3;
}
const gen = generator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

---

**97. What is the purpose of the bind() method in JavaScript?**

**Answer:**
bind() creates a new function with this set to a specific value.

Example:

```
const person = {
    name: "John",
    greet: function() {
        console.log(`Hello, ${this.name}`);
    }
};

const greetFunc = person.greet.bind(person);
greetFunc(); // "Hello, John"
```

---

Copyright © 2025 Skill Foundry

**98. What is the call() and apply() method in JavaScript?**

**Answer:**

- **call()**: Calls a function with a specified this value and arguments separately.
- **apply()**: Calls a function with this value and arguments as an array.

Example:

```
function greet(message) {
    console.log(`${message}, ${this.name}`);
}

const user = { name: "Alice" };

greet.call(user, "Hello"); // "Hello, Alice"
greet.apply(user, ["Hi"]); // "Hi, Alice"
```

---

**99. What is the difference between Object.freeze() and Object.seal()?**

**Answer:**

- **Object.freeze()**: Prevents adding, deleting, or modifying properties.
- **Object.seal()**: Prevents adding or deleting properties but allows modification.

Example:

```
const obj = { name: "John" };

Object.freeze(obj);
obj.name = "Alice"; // No effect
console.log(obj.name); // "John"
```

Copyright © 2025 Skill Foundry

```
const obj2 = { age: 25 };
Object.seal(obj2);
obj2.age = 30; // Allowed
delete obj2.age; // Not allowed
console.log(obj2.age); // 30
```

---

**100. What is memoization in JavaScript?**

**Answer:**
Memoization is an optimization technique to cache function results for faster performance.

Example:

```
function memoize(fn) {
    let cache = {};
    return function(...args) {
        let key = JSON.stringify(args);
        if (cache[key]) return cache[key];
        cache[key] = fn(...args);
        return cache[key];
    };
}

const factorial = memoize(n => (n <= 1 ? 1 : n *
factorial(n - 1)));

console.log(factorial(5)); // 120 (computed)
console.log(factorial(5)); // 120 (cached)
```

Copyright © 2025 Skill Foundry

**101. What is event delegation in JavaScript?**

**Answer:**
Event delegation is a technique where a parent element handles events for its child elements using event bubbling. This improves performance by reducing the number of event listeners.

Example:

```
document.getElementById("parent").addEventListener("click", function(event) {
    if (event.target && event.target.matches("button")) {
        console.log("Button clicked:", event.target.textContent);
    }
});
```

This ensures all dynamically added buttons inside #parent will be handled.

---

**102. What is the difference between window.onload and DOMContentLoaded?**

**Answer:**

- **window.onload** fires when the entire page, including images and styles, is fully loaded.
- **DOMContentLoaded** fires when the HTML is fully parsed but before images/styles are loaded.

Example:

```
document.addEventListener("DOMContentLoaded", () => {
    console.log("DOM fully loaded");
});


window.onload = () => {
    console.log("Entire page loaded");
```

Copyright © 2025 Skill Foundry

```
};
```

---

**103. How does JavaScript handle asynchronous operations?**

**Answer:**
JavaScript uses the **event loop**, **callbacks**, **Promises**, and **async/await** to handle asynchronous operations.

Example using **Promises**:

```
function fetchData() {
    return new Promise(resolve => {
        setTimeout(() => resolve("Data loaded"),
2000);
    });
}

fetchData().then(console.log); // Data loaded
(after 2s)
Example using async/await:
javascript
CopyEdit
async function fetchData() {
    let data = await new Promise(resolve =>
setTimeout(() => resolve("Data loaded"), 2000));
    console.log(data);
}

fetchData();
```

---

Copyright © 2025 Skill Foundry

**104. What are WeakMap and WeakSet in JavaScript?**

**Answer:**

- **WeakMap**: A special Map where keys must be objects and are weakly referenced (garbage-collected when not used).
- **WeakSet**: A Set that holds weak references to objects.

Example:

```
let weakMap = new WeakMap();
let obj = {};
weakMap.set(obj, "Hello");
console.log(weakMap.get(obj)); // "Hello"
obj = null; // Now it will be garbage collected
```

---

**105. How does JavaScript handle floating-point precision issues?**

**Answer:**
JavaScript follows IEEE 754 for floating-point arithmetic, which may cause precision errors.

Example of a precision issue:

```
console.log(0.1 + 0.2); // 0.30000000000000004
```

Solution using toFixed():

```
console.log((0.1 + 0.2).toFixed(2)); // "0.30"
```

Solution using Number.EPSILON:

```
function preciseSum(a, b) {
    return Math.round((a + b + Number.EPSILON) *
100) / 100;
}
console.log(preciseSum(0.1, 0.2)); // 0.3
```

Copyright © 2025 Skill Foundry